



Universidad de Guadalajara

CUCEI

Centro Universitario de Ciencias Exactas e Ingenierías.

PROGRAMACIÓN ESTRUCTURADA

Profesor: JORGE ERNESTO LOPEZ ARCE DELGADO

Rocha Perez Dinora Marlen

Sergio Josben Chávez Lupercio

Paola Sofia Sainz Barrios

Juan Pablo Jimenez Alcaraz

Introducción

Nuestro código consiste en la implementación del juego Battleship, el cual se basa en un sistema de coordenadas, donde el objetivo se basa en hundir la flota del jugador enemigo. Nuestro programa en lenguaje C, se basa en la implementación de iteraciones de ciclos for, para expresar de esta manera las mediante matrices del tablero del juego, así como la implementación de distintas validaciones para conseguir las posiciones de los barcos de los jugadores y las coordenadas que ingresen los usuarios.

De esta manera el objetivo de nuestro juego es desarrollar un código funcional, Optimizando el recorrido de matrices bidimensionales mediante ciclos for anidados para el tablero de los jugadores que es en esencia un plano ordenado e implementando algoritmos de búsqueda y comparación utilizando iteraciones para verificar si un disparo ha impactado en un barco o en agua. Así como estructuras de control condicionales e iterativas, para determinar las distintas decisiones de los jugadores y que el programa actúe de acuerdo a estas. También implementamos un menú switch en el cual el usuario puede seleccionar leer las reglas del juego o comenzar con una partida, dándole orden al juego y una interfaz de usuario apropiada. También nuestro programa realiza múltiples validaciones para que los jugadores ingresen valores ciertos y/o legales para que el programa no se interrumpa ante entradas de datos inválidas, haciendo de nuevo esta iteración del ciclo hasta obtener un valor válido.

Finalmente las estructuras de control empleadas en este código. La continuidad de la partida es lograda mediante una estructura de control iterativa do while, la cual permite que el programa se ejecute de forma indefinida hasta que una estructura condicional detecta que el contador de barcos enemigos ha llegado a cero y con un jugador ganador.

Links de repositorios de github

Rocha Perez Dinora Marlen

<https://github.com/dinorarochoa8332-byte/ACTIVIDADES-PROGRAMACION-ESTRUCTURADA>

Sergio Josben Chávez Lupercio

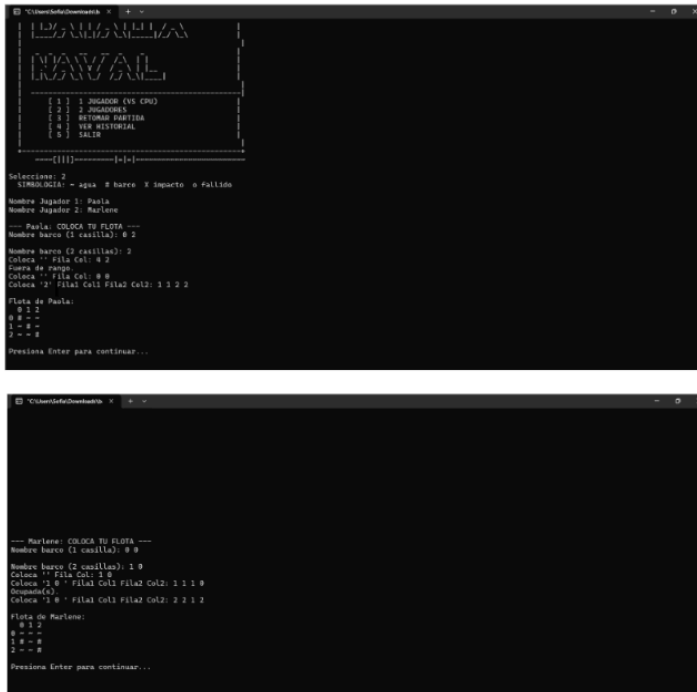
<https://github.com/sJosben/Actividades-de-programacion/tree/main/JuegoGuerraNaval>

Paola Sofia Sainz Barrios:

<https://github.com/paolasainz6547-commits/Arreglos-de-Caracteres-.git>

Juan Pablo Jimenez Alcaraz: [P.Estructurada-JP/Battleship at main · juanjimenez6916-web/P.Estructurada-JP](#)

Capturas



```

BATTLESHIP
NAVAL

[ 1 ] 1 JUGADOR (VS CPU)
[ 2 ] 2 JUGADORES
[ 3 ] 3 JUGADORES
[ 4 ] 4 JUGADORES
[ 5 ] 5 JUGADORES
[ 6 ] 6 JUGADORES
[ 7 ] 7 JUGADORES
[ 8 ] 8 JUGADORES
[ 9 ] 9 JUGADORES
[10] 10 JUGADORES

-----[10]-----[10]-----

Selecciona: 2
1 JUGADOR (VS CPU)
2 JUGADORES
3 JUGADORES
4 JUGADORES
5 JUGADORES
6 JUGADORES
7 JUGADORES
8 JUGADORES
9 JUGADORES
10 JUGADORES

Nombre Jugador 1: Paola
Nombre Jugador 2: Marlene

----- Paola: COLOCA TU FLETA -----
Nombre barco (1 casilla): 0 2
Nombre barco (2 casillas): 2
Columna: 1 Fila Col: 0 2
Punto de origen:
Columna: 1 Fila Col: 0 8
Columna: 2 Fila Col: 1 1 2 2

Fleto de Paola:
0 1 2
0 0 0
1 0 0
2 0 0

Presiona Enter para continuar...

----- Marlene: COLOCA TU FLETA -----
Nombre barco (1 casilla): 0 9
Nombre barco (2 casillas): 1 9
Columna: 1 Fila Col: 1 8
Columna: 2 Fila Col: 1 1 1 9
Columna: 3 Fila Col: 1 1 1 2
Columna: 4 Fila Col: 2 1 1 2

Fleto de Marlene:
0 1 2
0 0 0
1 0 0
2 0 0

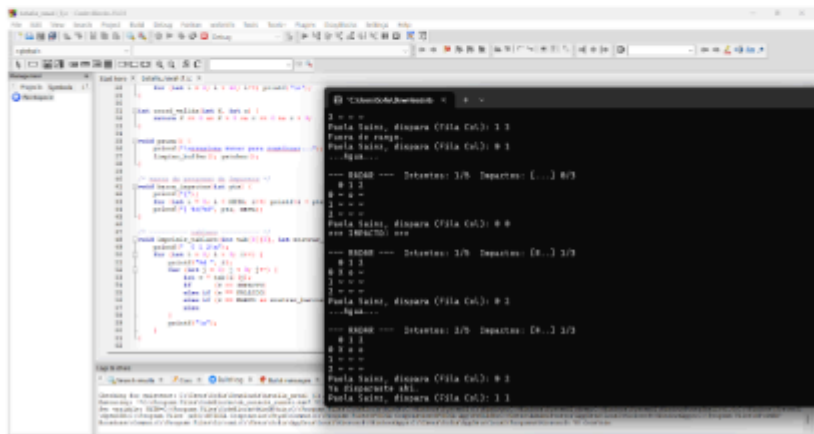
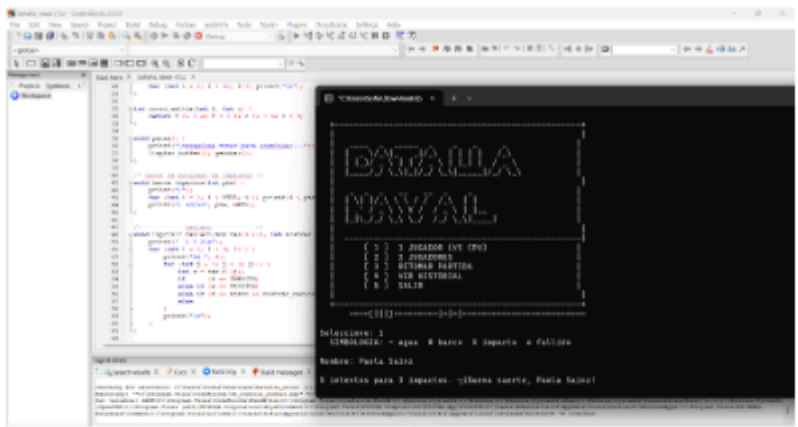
Presiona Enter para continuar...
```

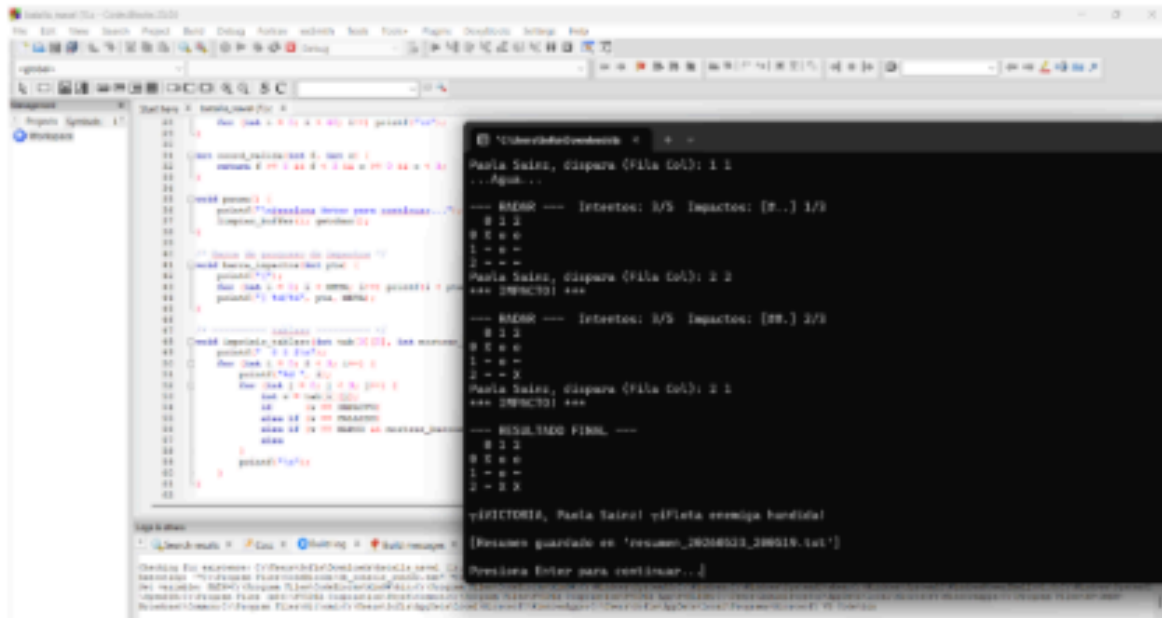
```
===== TURNO DE Paola =====
Impactos: [00.] 2/3
Buscando: ' ' y '1 0 '
TU FLOTA (Paola)      RADAR (Marlene)
0 1 2      0 1 2
0 0 - - -      0 - 0 - -
1 - X 0      1 X 0 - -
2 0 - X      2 - - X
Atacar (Fila Col): 0 1
Va disparando ahí:
Atacar (Fila Col): 1 0
Va disparando ahí:
Atacar (Fila Col): 1 2
*** IMPACTO! ***

===== FIN =====
~!~!RADAR: Paola!
Paola: 3 impactos | Marlene: 2 impactos
[Resumen guardado en 'resumen_20260523_201053.txt']
Presiona Enter para continuar...

+-----+
| BATA|
| LLA|
|+-----+
| NAVAL|
|+-----+
|
| [ 1 ] 1 JUGADOR (VS CPU)
| [ 2 ] 2 JUGADORES
| [ 3 ] RETOMAR PARTIDA
|+-----+

=====
```





Código de Videojuego

```
C/C++
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_PARTIDAS 50
#define AGUA 0
#define BARCO 1
#define FALLIDO 2
#define IMPACTO 3
#define META 3

struct Partida {
    char id[30], j1[20], j2[20];
    char b1j1[20], b2j1[20], b1j2[20], b2j2[20];
    int T1[3][3], T2[3][3];
    int p1, p2, terminada;
    int turno_actual; /* 0=J1, 1=J2 */
};

/* ----- utilidades ----- */
void limpiar_buffer() {
    int ch;
    while ((ch = getchar()) != '\n' && ch != EOF);
}

void limpiar_pantalla() {
    for (int i = 0; i < 40; i++) printf("\n");
}

int coord_valida(int f, int c) {
    return f >= 0 && f < 3 && c >= 0 && c < 3;
}

void pausa() {
    printf("\nPresiona Enter para continuar...");
    limpiar_buffer(); getchar();
}

/* Barra de progreso de impactos */
void barra_impactos(int pts) {
    printf("[");
    for (int i = 0; i < META; i++) printf(i < pts ? "#" : ".");
    printf("] %d/%d", pts, META);
}
```

```

/* ----- tablero ----- */
void imprimir_tablero(int tab[3][3], int mostrar_barcos) {
    printf("  0 1 2\n");
    for (int i = 0; i < 3; i++) {
        printf("%d ", i);
        for (int j = 0; j < 3; j++) {
            int v = tab[i][j];
            if (v == IMPACTO)                printf("X ");
            else if (v == FALLIDO)           printf("o ");
            else if (v == BARCO && mostrar_barcos) printf("# ");
            else                             printf("~ ");
        }
        printf("\n");
    }
}

/* Muestra ambos tableros lado a lado (propio visible, enemigo oculto) */
void imprimir_doble(int propio[3][3], int enemigo[3][3],
                    char nom_p[20], char nom_e[20]) {
    printf("  TU FLOTA (%s)    RADAR (%s)\n", nom_p, nom_e);
    printf("  0 1 2          0 1 2\n");
    for (int i = 0; i < 3; i++) {
        printf("%d ", i);
        for (int j = 0; j < 3; j++) {
            int v = propio[i][j];
            printf("%c ", v==IMPACTO?'X': v==FALLIDO?'o': v==BARCO?'#':'~');
        }
        printf("      %d ", i);
        for (int j = 0; j < 3; j++) {
            int v = enemigo[i][j];
            printf("%c ", v==IMPACTO?'X': v==FALLIDO?'o':'~');
        }
        printf("\n");
    }
}

/* ----- archivos ----- */
void log_movimiento(char pid[30], char jugador[20], int f, int c, const char
*res) {
    FILE *a = fopen("movimientos.txt", "a");
    if (!a) return;
    time_t t = time(NULL);
    char hora[20];
    strftime(hora, sizeof(hora), "%H:%M:%S", localtime(&t));
    fprintf(a, "[%s] %s | %s | (%d,%d) | %s\n", hora, pid, jugador, f, c, res);
    fclose(a);
}

```

```

void guardar_partida(struct Partida *p) {
    struct Partida lista[MAX_PARTIDAS];
    int total = 0, idx = -1;
    FILE *a = fopen("partidas.dat", "rb");
    if (a) {
        while (fread(&lista[total], sizeof(struct Partida), 1, a)==1 &&
total<MAX_PARTIDAS)
            total++;
        fclose(a);
    }
    for (int i = 0; i < total; i++)
        if (strcmp(lista[i].id, p->id)==0) { idx = i; break; }
    if (idx >= 0) lista[idx] = *p;
    else if (total < MAX_PARTIDAS) lista[total++] = *p;

    a = fopen("partidas.dat", "wb");
    if (a) { for (int i=0;i<total;i++) fwrite(&lista[i], sizeof(struct
Partida), 1, a); fclose(a); }
}

void exportar_resumen(struct Partida *p, const char *ganador) {
    char nombre[60];
    sprintf(nombre, "resumen_%s.txt", p->id);
    FILE *a = fopen(nombre, "w");
    if (!a) return;
    fprintf(a, "=====\n");
    fprintf(a, "          BATALLA NAVAL - RESUMEN\n");
    fprintf(a, "=====\n");
    fprintf(a, "ID          : %s\n", p->id);
    fprintf(a, "J1          : %s Impactos: %d/%d\n", p->j1, p->p1, META);
    fprintf(a, "J2          : %s Impactos: %d/%d\n", p->j2, p->p2, META);
    fprintf(a, "Ganador     : %s\n", ganador);
    for (int t = 0; t < 2; t++) {
        int (*tab)[3] = t==0 ? p->T1 : p->T2;
        fprintf(a, "\nTablero final de %s:\n 0 1 2\n", t==0?p->j1:p->j2);
        for (int i=0;i<3;i++) {
            fprintf(a, "%d ", i);
            for (int j=0;j<3;j++) {
                int v = tab[i][j];
                fprintf(a, "%c ", v==BARCO?'#': v==IMPACTO?'X':
v==FALLIDO?'o':'~');
            }
            fprintf(a, "\n");
        }
    }
    fprintf(a, "=====\n");
    fclose(a);
}

```



```

    printf("\n[Resumen guardado en '%s']\n", nombre);
}

/* ----- historial de partidas ----- */
void ver_historial() {
    struct Partida lista[MAX_PARTIDAS];
    int total = 0;
    FILE *a = fopen("partidas.dat", "rb");
    if (!a) { printf("\nSin historial.\n"); return; }
    while (fread(&lista[total], sizeof(struct Partida), 1, a) == 1 &&
total<MAX_PARTIDAS) total++;
    fclose(a);
    if (total == 0) { printf("\nSin historial.\n"); return; }
    printf("\n--- HISTORIAL (%d partida(s)) ---\n", total);
    for (int i = 0; i < total; i++) {
        printf("[%s] %s vs %s | %d-%d | %s\n",
            lista[i].id, lista[i].j1, lista[i].j2,
            lista[i].p1, lista[i].p2,
            lista[i].terminada ? "TERMINADA" : "EN CURSO");
    }
}

/* ----- colocación de barcos ----- */
void colocar_barcos(int tab[3][3], char nom[20], char b1[20], char b2[20]) {
    int f, c, f2, c2;
    printf("\n--- %s: COLOCA TU FLOTA ---\n", nom);

    printf("Nombre barco (1 casilla): ");
    limpiar_buffer();
    fgets(b1, 20, stdin);
    b1[strcspn(b1, "\n")] = 0;

    printf("Nombre barco (2 casillas): ");
    fgets(b2, 20, stdin);
    b2[strcspn(b2, "\n")] = 0;

    while (1) {
        printf("Coloca '%s' Fila Col: ", b1);
        if (scanf("%d %d", &f, &c) != 2) { limpiar_buffer(); continue; }
        if (!coord_valida(f, c)) { printf("Fuera de rango.\n"); continue; }

        if (tab[f][c]) { printf("Ocupada.\n"); continue; }
        tab[f][c] = BARCO; break;
    }

    while (1) {
        printf("Coloca '%s' Fila1 Col1 Fila2 Col2: ", b2);
        if (scanf("%d %d %d %d", &f, &c, &f2, &c2) != 4) { limpiar_buffer();
continue; }

```



```

printf(" | |_/\\_/\\_/ \\_\\ \\_\\_/ /_/ \\_\\|____| |\\n");
printf(" | |\\n");
printf(" | -----|\\n");
printf(" | [ 1 ] 1 JUGADOR (VS CPU) |\\n");
printf(" | [ 2 ] 2 JUGADORES |\\n");
printf(" | [ 3 ] RETOMAR PARTIDA |\\n");
printf(" | [ 4 ] VER HISTORIAL |\\n");
printf(" | [ 5 ] SALIR |\\n");
printf(" | |\\n");
printf(" +-----+\\n");
printf("      ~~~[||]~~~~~|=|=|~~~~~\\n\\n");
}

/* ----- simbología ----- */
void simbologia() {
    printf(" SIMBOLOGIA: ~ agua # barco X impacto o fallido\\n\\n");
}

/* ----- modo 1 jugador ----- */
void modo_1_jugador() {
    char jugador[20];
    int tablero[3][3] = {0};
    int intentos = 0, impactos = 0;
    char pid[30];
    struct Partida p;

    simbologia();
    printf("Nombre: ");
    limpiar_buffer();
    fgets(jugador, sizeof(jugador), stdin);
    jugador[strcspn(jugador, "\\n")] = 0;

    /* CPU coloca barcos en posiciones fijas */
    tablero[0][0] = BARCO;
    tablero[2][1] = BARCO;
    tablero[2][2] = BARCO;

    strftime(pid, sizeof(pid), "%Y%m%d_%H%M%S",
    localtime(&(time_t){time(NULL)}));

    printf("\\n5 intentos para 3 impactos. ¡Buena suerte, %s!\\n", jugador);

    while (intentos < 5 && impactos < META) {
        printf("\\n--- RADAR --- Intentos: %d/5 Impactos: ", intentos);
        barra_impactos(impactos); printf("\\n");
        imprimir_tablero(tablero, 0);

        int f, c;

```

```

while (1) {
    printf("%s, dispara (Fila Col): ", jugador);
    if (scanf("%d %d",&f,&c)!=2) { limpiar_buffer(); continue; }
    if (!coord_valida(f,c)) { printf("Fuera de rango.\n"); continue; }
    if (tablero[f][c]==FALLIDO||tablero[f][c]==IMPACTO) { printf("Ya
disparaste ahí.\n"); continue; }
    break;
}

if (tablero[f][c] == BARCO) {
    tablero[f][c] = IMPACTO; impactos++;
    printf("*** IMPACTO! ***\n");
    log_movimiento(pid, jugador, f, c, "IMPACTO");
} else {
    tablero[f][c] = FALLIDO; intentos++;
    printf("...Agua...\n");
    log_movimiento(pid, jugador, f, c, "FALLIDO");
}
}

printf("\n--- RESULTADO FINAL ---\n");
imprimir_tablero(tablero, 1);

if (impactos == META)
    printf("\n¡VICTORIA, %s! ¡Flota enemiga hundida!\n", jugador);
else
    printf("\nDerrota. Los barcos estaban en [0,0], [2,1] y [2,2].\n");

strcpy(p.id, pid); strcpy(p.j1, jugador); strcpy(p.j2, "CPU");
for (int i=0;i<3;i++) for(int j=0;j<3;j++) { p.T1[i][j]=tablero[i][j];
p.T2[i][j]=0; }
p.p1=impactos; p.p2=0; p.terminada=1;
guardar_partida(&p);
exportar_resumen(&p, impactos==META ? jugador : "CPU");
}

/* ----- modo 2 jugadores ----- */
void modo_2_jugadores() {
    char nom[2][20];
    char b1[2][20], b2[2][20];
    int T[2][3][3] = {{{0}}};
    int pts[2] = {0, 0};
    char pid[30];
    struct Partida partida;

    simbologia();
    limpiar_buffer();
    for (int i = 0; i < 2; i++) {

```

```

        printf("Nombre Jugador %d: ", i+1);
        fgets(nom[i], sizeof(nom[i]), stdin);
        nom[i][strcspn(nom[i], "\n")] = 0;
    }

    strptime(pid, sizeof(pid), "%Y%m%d_%H%M%S",
    localtime(&(time_t){time(NULL)}));

    /* Fase de colocación */
    for (int i = 0; i < 2; i++) {
        colocar_barcos(T[i], nom[i], b1[i], b2[i]);
        pausa(); limpiar_pantalla();
    }

    strcpy(partida.id, pid);
    strcpy(partida.j1, nom[0]); strcpy(partida.j2, nom[1]);
    strcpy(partida.b1j1, b1[0]); strcpy(partida.b2j1, b2[0]);
    strcpy(partida.b1j2, b1[1]); strcpy(partida.b2j2, b2[1]);
    partida.terminada = 0; partida.turno_actual = 0;

    /* Bucle de juego */
    while (pts[0] < META && pts[1] < META) {
        for (int cur = 0; cur < 2 && pts[0]<META && pts[1]<META; cur++) {
            int ene = 1 - cur;
            printf("\n===== TURNO DE %s =====\n", nom[cur]);
            printf("Impactos: "); barra_impactos(pts[cur]); printf("\n");
            printf("Buscando: '%s' y '%s'\n", b1[ene], b2[ene]);
            imprimir_doble(T[cur], T[ene], nom[cur], nom[ene]);

            disparar(T[ene], nom[cur], pid, &pts[cur]);

            /* Guardar progreso tras cada disparo */
            for (int i=0;i<3;i++) for(int j=0;j<3;j++) {
                partida.T1[i][j]=T[0][i][j]; partida.T2[i][j]=T[1][i][j];
            }
            partida.p1=pts[0]; partida.p2=pts[1];
            partida.turno_actual = (pts[0]<META && pts[1]<META) ? (1-cur) :
cur;

            guardar_partida(&partida);

            if (pts[cur] < META) { pausa(); limpiar_pantalla(); }
        }
    }

    char *ganador = pts[0]==META ? nom[0] : nom[1];
    printf("\n===== FIN =====\n");
    printf(";GANADOR: %s!\n", ganador);
    printf("%s: %d impactos | %s: %d impactos\n", nom[0],pts[0],nom[1],pts[1]);

```

```

    partida.terminada = 1;
    guardar_partida(&partida);
    exportar_resumen(&partida, ganador);
}

/* ----- retomar partida ----- */
void retomar_partida() {
    struct Partida lista[MAX_PARTIDAS];
    int total = 0, count = 0, indices[MAX_PARTIDAS];
    FILE *a = fopen("partidas.dat", "rb");
    if (!a) { printf("\nNo hay partidas guardadas.\n"); return; }
    while (fread(&lista[total], sizeof(struct Partida), 1, a) == 1 &&
total<MAX_PARTIDAS) total++;
    fclose(a);

    printf("\n--- PARTIDAS EN CURSO ---\n");
    for (int i = 0; i < total; i++)
        if (!lista[i].terminada && strcmp(lista[i].j2, "CPU") != 0) {
            printf("[%d] %s | %s vs %s | %d-%d | Turno: %s\n",
                count+1, lista[i].id, lista[i].j1, lista[i].j2,
                lista[i].p1, lista[i].p2,
                lista[i].turno_actual==0 ? lista[i].j1 : lista[i].j2);
            indices[count++] = i;
        }
    if (count == 0) { printf("No hay partidas para retomar.\n"); return; }

    int sel;
    printf("\nSelecciona (1-%d) o 0 para cancelar: ", count);
    scanf("%d", &sel);
    if (sel < 1 || sel > count) return;

    struct Partida *p = &lista[indices[sel-1]];
    char nom[2][20];
    int T[2][3][3];
    int pts[2] = {p->p1, p->p2};
    strcpy(nom[0], p->j1); strcpy(nom[1], p->j2);
    for (int i=0; i<3; i++) for (int j=0; j<3; j++) { T[0][i][j]=p->T1[i][j];
T[1][i][j]=p->T2[i][j]; }

    printf("\nRetomando: %s vs %s | Impactos %d-%d | Turno de %s\n",
        nom[0], nom[1], pts[0], pts[1],
        p->turno_actual==0 ? nom[0] : nom[1]);
    simbologia();

    /* Continuar desde el turno guardado */
    int inicio = p->turno_actual;
    while (pts[0] < META && pts[1] < META) {

```

```

    for (int i = 0; i < 2 && pts[0]<META && pts[1]<META; i++) {
        int cur = (inicio + i) % 2;
        int ene = 1 - cur;
        printf("\n===== TURNO DE %s =====\n", nom[cur]);
        printf("Impactos: "); barra_impactos(pts[cur]); printf("\n");
        imprimir_doble(T[cur], T[ene], nom[cur], nom[ene]);

        disparar(T[ene], nom[cur], p->id, &pts[cur]);

        for (int r=0;r<3;r++) for(int c2=0;c2<3;c2++) {
p->T1[r][c2]=T[0][r][c2]; p->T2[r][c2]=T[1][r][c2]; }
        p->p1=pts[0]; p->p2=pts[1];
        p->turno_actual = (pts[0]<META && pts[1]<META) ? (1-cur) : cur;
        guardar_partida(p);

        if (pts[cur] < META) { pausa(); limpiar_pantalla(); }
    }
    inicio = 0;
}

char *ganador = pts[0]==META ? nom[0] : nom[1];
printf("\n=== FIN === ;GANADOR: %s!\n", ganador);
p->terminada = 1; guardar_partida(p);
exportar_resumen(p, ganador);
}

/* ----- main ----- */
int main() {
    int op;
    do {
        portada();
        printf("Seleccione: ");
        if (scanf(" %d", &op) != 1) { limpiar_buffer(); continue; }
        switch (op) {
            case 1: modo_1_jugador(); break;
            case 2: modo_2_jugadores(); break;
            case 3: retomar_partida(); break;
            case 4: ver_historial(); break;
            case 5: printf("\n ;Hasta la proxima, Almirante!\n\n"); break;
            default: printf("\nOpcion invalida.\n"); break;
        }
        if (op != 5) { pausa(); }
    } while (op != 5);
    return 0;
}

```

Conclusión

Al desarrollar este juego de Batalla Naval, logramos poner en práctica y aterrizar conceptos fundamentales de la programación en C. Principalmente, aprendimos a estructurar un programa interactivo utilizando un ciclo continuo combinado con un menú de opciones. También comprendimos cómo manejar la información espacial mediante el uso de arreglos bidimensionales, es decir, matrices. Esta herramienta fue clave para crear un tablero virtual de filas y columnas donde pudimos guardar, leer y modificar las posiciones de los barcos, así como el registro de los disparos. Pusimos en práctica el uso y la importancia de los strings al poder guardar los nombres de los jugadores y de los propios barcos.

Finalmente, desarrollamos nuestra lógica de programación al usar condicionales para darle reglas e "inteligencia" al juego. Aprendimos a hacer que el código evalúe cada coordenada ingresada por el usuario para decidir y mostrar visualmente si el disparo cayó al agua, si acertó a un barco o si esa casilla ya había sido elegida previamente. Todo esto, sumado al control de turnos, me enseñó cómo conectar diferentes instrucciones básicas para construir una experiencia completa y funcional desde cero. En conclusión fue una experiencia bastante diferente que reto nuestra lógica en programación en C :).